POLITECHNIKA BIAŁOSTOCKA

WYDZIAŁ INFORMATYKI

DOKUMENTACJA PROJEKTOWA

TEMAT: Detekcja tekstu wygenerowanego AI

WYKONAWCY: Michał Dąbrowski
Dominik Zabielski
Michał Niechciałkowski

PROWADZĄCY: dr hab. inż. Sławomir Zieliński

BIAŁYSTOK, LISTOPAD 2026

Subject of diploma thesis:
Detection of AI-generated text

Summary

This document presents the project plan for detecting AI-generated text. It covers related work, consultations with an expert (dr hab. Piotr Hońko) and with a generative AI system, a brief requirements analysis, project scope, a preliminary schedule, and a GenAI usage statement. The minimum success criterion is classification accuracy of at least 51% on a chosen test set.

Spis treści

1 Opis zmian **5**

2 Tło projektu **6**

2.1 Przegląd podobnych rozwiązań 6

2.2 Rozwój metod detekcji tekstu generowanego maszynowo 6

2.2.1 Wczesne metody klasyfikacji i statystyki rangowe (2019–2021) . . 6

2.2.2 Wykorzystanie miar zakłopotania i zmienności stylu wypowiedzi (2022–2023) 7

2.2.3 Beznadzorowa analiza lokalnej krzywizny prawdopodobieństwa (2023) 8

2.2.4 Koncepcja znakowania wodnego w procesie generacji tekstu (2023) 8

2.2.5 Weryfikacja ograniczeń, stronniczości i odporności na modyfikacje tekstu (2023–2024) 9

2.2.6 Detekcja modeli nowej generacji i wnioskowanie z użyciem łańcucha myśli (2024–2026) 10

2.2.7 Regulacje UE i komercyjne znakowanie wodne tekstu (2026) . . . 10

2.2.8 Zestawienie problemów badawczych i ewolucji rozwiązań 11

2.3 Stan wiedzy 13

2.4 Uwarunkowania komercyjne 13

2.5 Wyniki konsultacji z ekspertem 13

2.6 Wyniki konsultacji AI 14

2.7 Porównanie odpowiedzi eksperta i AI 15

3 Skrótowa analiza wymagań **16**

3.1 Wymagania funkcjonalne 16

3.2 Wymagania niefunkcjonalne 17

4 Zakres projektu i harmonogram **18**

4.1 Zakres 18

4.2 Proponowane zbiory danych 18

4.3 Kamienie milowe 19

4.4 Harmonogram (diagram Gantta) 19

5 Szczegółowa specyfikacja projektu i opis wymagań 20

5.1 Cel i kryterium sukcesu 20

5.2 Wejście / wyjście 20

5.3 Przepływ przetwarzania 20

5.4 Założenia 21

5.5 Ograniczenia 21

5.6 Plan ewaluacji 21

5.7 Środowisko realizacji 22

6 Opis wyników prac 23

6.1 Ścieżka machine learning 23

7 Oświadczenie wykorzystania narzędzi GenAI 26

Bibliografia 27

Spis rysunków 27

1 Opis zmian

Wersja	Data	Opis zmian	Autorzy
1.0	20.10.2026	Pierwsza wersja dokumentacji: tło, konsultacje ekspert/AI, wymagania, zakres i harmonogram.	M. Dąbrowski, D. Zabielski
2.0	27.10.2026	Rozwinięcie rozdziału drugiego - Tło Projektu.	M. Dąbrowski, D. Zabielski, M. Niechciałkowski
3.0	03.11.2026	Rozszerzenie tła o textGrain / EU text provenance (OpenAI); uszczegółowienie wymagań, zakresu, Gantta i specyfikacji.	M. Dąbrowski, D. Zabielski, M. Niechciałkowski

2 Tło projektu

2.1 Przegląd podobnych rozwiązań

Detekcja tekstu generowanego przez modele językowe (LLM) jest aktywnie rozwijanym obszarem. Istnieją zarówno narzędzia komercyjne, jak i metody badawcze:

- **GPTZero, Turnitin, Originality.ai** - systemy komercyjne stosowane m.in. w edukacji; opierają się na cechach stylometrycznych i modelach klasyfikacji.
- **DetectGPT / Fast-DetectGPT** - metody oparte na zaburzeniach prawdopodobieństwa (perturbation) tekstu względem modelu językowego.
- **Klasyfikatory nadzorowane** - modele typu RoBERTa/BERT trenowane na parach tekstów ludzkich i sztucznych (np. HC3, M4).
- **Watermarking** - oznaczanie wyjścia LLM na etapie generacji (np. schemat Kirchenbauer i in., komercyjne textGrain OpenAI); wymaga współpracy po stronie generatora i nie obejmuje modeli zewnętrznych.

W praktyce dokładność detektorów spada przy krótkich tekstach, parafrazie oraz tekstach wielojęzycznych. Dla języka polskiego liczba gotowych narzędzi i zbiorów jest wyraźnie mniejsza niż dla angielskiego.

2.2 Rozwój metod detekcji tekstu generowanego maszynowo

Historia badań nad identyfikacją tekstu generowanego przez sztuczną inteligencję stanowi ciąg logicznie powiązanych odkryć, w których publikacja każdego kolejnego rozwiązania wynikała bezpośrednio z ograniczeń, błędów lub barier obliczeniowych zidentyfikowanych u poprzedników.

2.2.1 Wczesne metody klasyfikacji i statystyki rangowe (2019–2021)

Początek systematycznych badań nad detekcją tekstu generowanego przez współczesne architektury transformera nastąpił wraz z zaprezentowaniem przez OpenAI modelu GPT-2 w 2019 roku. Obawy o możliwość masowego generowania dezinformacji skłoniły badaczy

z zespołu OpenAI, Solaimana i współpracowników [**solaiman2019release**], do opracowania pierwszych klasyfikatorów nadzorowanych. Wykorzystali oni dostrajanie modelu koderowego RoBERTa na parach tekstów ludzkich oraz syntetycznych wygenerowanych przez GPT-2. Podejście to, choć skuteczne w wąskiej domenie tekstów treningowych, ujawniło podstawową wadę modeli nadzorowanych: wysoki koszt pozyskiwania reprezentatywnych danych oraz gwałtowny spadek skuteczności po zmianie dziedziny lub stylu wypowiedzi.

Odpowiedzią na tę niedogodność była praca Gehrmann, Strobelta i Rusha z uniwersytetu Harvarda [**gehrmann2019gltr**]. Badacze ci zauważyli, że zamiast trenować kosztowne klasyfikatory, można badać bezpośrednio zachowanie probabilistyczne modeli językowych. Opracowali oni narzędzie GLTR, opierając się na hipotezie, że automatyczne generatory dążą do maksymalizacji wiarygodności lokalnej i próbkują słowa z czołówki rankingu predykcji, podczas gdy autorzy ludzcy znacznie częściej sięgają po słowa rzadsze statystycznie. Podejście to pozwalało na beznadzorową analizę tekstów, jednak wraz z upowszechnieniem nowych algorytmów dekodowania, takich jak próbkowanie jądrowe (*nucleus sampling*), rozkłady rang uległy zatarciu, co wymusiło poszukiwanie bardziej złożonych metryk.

2.2.2 Wykorzystanie miar zakłopotania i zmienności stylu wypowiedzi (2022–2023)

Upublicznienie modelu ChatGPT pod koniec 2022 roku doprowadziło do masowego wykorzystania generacji tekstu w środowisku akademickim i zawodowym, uniemożliwiając poleganie na wczesnych heurystykach. Na to wyzwanie odpowiedział Edward Tian [**gptzero2023**], tworząc narzędzie GPTZero. Tian zauważył, że sama wartość zakłopotania (*perplexity*) pomocniczego modelu językowego nie pozwala na jednoznaczne rozróżnienie autorstwa w tekstach o zróżnicowanym rejestrze. Zaproponował więc połączenie miary zakłopotania z analizą dynamiki wypowiedzi, określaną jako *burstiness*. Koncepcja Tiana opierała się na założeniu, że człowiek naturalnie moduluje tempo tekstu, przeplatając zdania złożone z konstrukcjami krótkimi i eliptycznymi, podczas gdy generatory LLM tworzą wypowiedzi o wyrównanym rytmie składniowym i stabilnym zakłopotaniu.

Założenia te legły u podstaw komercyjnych detektorów (takich jak Turnitin, Copyleaks czy Originality.ai), jednak wymagały one niezależnej weryfikacji naukowej. Zadania tego podjął się zespół Guo i współpracowników [**guo2023hc3**], tworząc zbiór badawczy HC3. Porównując tysiące odpowiedzi ekspertów z generacjami modelu ChatGPT w dziedzinach prawa, medycyny i finansów, autorzy potwierdzili większą poprawność gramatyczną oraz

formalizm generacji maszynowych, ale jednocześnie wykazali, że w dziedzinach wysoce sformalizowanych ludzkie wypowiedzi wykazują cechy stylometryczne niemal identyczne z modelami, co skutkuje wysokim odsetkiem błędnych klasyfikacji.

2.2.3 Beznadzorowa analiza lokalnej krzywizny prawdopodobieństwa (2023)

Niejednoznaczność cech stylometrycznych oraz nieprzejrzystość komercyjnych narzędzi skłoniły badaczy ze Stanford University pod kierunkiem Erica Mitchella [**mitchell2023detectgpt**] do poszukiwania głębszych sygnatur matematycznych. Mitchell i współpracownicy odkryli fundamentalną właściwość geometryczną: teksty wygenerowane przez dany model językowy znajdują się w lokalnych maksimach funkcji logarytmu prawdopodobieństwa tego generatora. Na tej podstawie zaproponowali metodę DetectGPT, która nie wymagała żadnego zbioru treningowego. Procedura polegała na nakładaniu drobnych modyfikacji semantycznych na badany tekst za pomocą pomocniczego modelu maskującego i obserwowaniu zachowania prawdopodobieństwa. Spadek wiarygodności po zniekształceniu tekstu świadczył o ujemnej krzywiźnie i wskazywał na generację sztuczną, podczas gdy dla tekstów ludzkich krzywizna pozostawała płaska.

Mimo przełomowego charakteru DetectGPT, metoda ta napotkała poważną barierę praktyczną: konieczność generowania dziesiątek wariantów tekstu sprawiała, że weryfikacja pojedynczego fragmentu trwała kilkadziesiąt sekund. Na ograniczenie to bezpośrednio odpowiedzieli Bao i współpracownicy [**bao2023fastdetectgpt**], publikując metodę Fast-DetectGPT. Autorzy wykazali matematycznie, że zamiast fizycznego generowania próbek perturbowanych w przestrzeni tekstowej, krzywiznę prawdopodobieństwa można wyznaczyć analitycznie poprzez warunkową wariancję prawdopodobieństwa na poziomie pojedynczych tokenów. Pozwoliło to osiągnąć ponad 300-krotne przyspieszenie czasu obliczeń przy zachowaniu, a w wielu przypadkach przewyższeniu trafności detekcji.

2.2.4 Koncepcja znakowania wodnego w procesie generacji tekstu (2023)

Część badaczy uznała jednak, że pasywna detekcja gotowego tekstu przypomina niekończący się wyścig technologiczny, w którym dalszy rozwój generatorów z czasem zniweczy skuteczność klasyfikatorów zewnętrznych. Wychodząc z tego założenia, Kirchenbauer i współpracownicy [**kirchenbauer2023watermark**] zaproponowali odmienne podejście, polegające na prewencyjnym znakowaniu wodnym (*watermarking*) w trakcie próbko-

wania słów przez sam generator. Na podstawie funkcji skrótu poprzedzającego tokena słownik dzielony jest pseudolosowo na listę faworyzowaną i zabronioną, co pozostawia w generowanym tekście statystyczną sygnaturę możliwą do bezbłędnej identyfikacji. Praca ta wywołała szeroką dyskusję w środowisku: choć metoda gwarantowała matematyczną pewność detekcji, jej praktyczne wdrożenie okazało się ograniczone do zamkniętych ekosystemów komercyjnych, nie rozwiązując problemu otwartych modeli językowych pobieranych lokalnie przez użytkowników.

2.2.5 Weryfikacja ograniczeń, stronniczości i odporności na modyfikacje tekstu (2023–2024)

Równolegle z rozwojem metod detekcji nasilała się fala badań weryfikujących ich rzeczywistą odporność w warunkach polowych. Przełomem w ocenie społecznych konsekwencji wdrażania detektorów była publikacja Lianga i współpracowników [**liang2023gptdetectors**]. Autorzy przetestowali popularne systemy na esejach egzaminacyjnych TOEFL napisanych przez osoby, dla których angielski nie był językiem ojczystym. Wykazano wówczas, że prostszy, powtarzalny dobór słownictwa autorów zagranicznych był przez algorytmy systematycznie utożsamiany z maszynowością, powodując ponad 61% fałszywych oskarżeń o użycie sztucznej inteligencji. Rezultaty te unaoczniły etyczne ryzyko stosowania detektorów jako narzędzi rozstrzygających.

Problem ten znalazł odzwierciedlenie w decyzji samej firmy OpenAI [**openai2023classifier**], która w lipcu 2023 roku oficjalnie zamknęła swoje narzędzie AI Text Classifier, motywując to niską czułością rzędu 26% i nadmierną liczbą fałszywych rozpoznań. Kropkę nad i postawiła praca teoretyczna Sadasivana i współpracowników [**sadasivan2023can**], w której dowiedziono, że w miarę jak dystrybucja prawdopodobieństwa generatorów zbliża się do dystrybucji mowy ludzkiej, pole pod krzywą ROC dowolnego detektora zmierza do poziomu losowego. Co więcej, wykazano, że celowe parafrazowanie tekstu za pomocą zewnętrznych narzędzi całkowicie zaciera ślady statystyczne wykorzystywane przez dotychczasowe metody.

2.2.6 Detekcja modeli nowej generacji i wnioskowanie z użyciem łańcucha myśli (2024–2026)

Najnowszy etap badań skupia się na odporności detektorów wobec modeli najnowszych generacji oraz celowych technik humanizacji wypowiedzi. W pracy Alshammari i Rao [**alshammari2025evaluating**] poddano weryfikacji teksty generowane przez model DeepSeek-V3 oraz ich zniekształcenia uzyskiwane za pomocą parafrazatorów takich jak QuillBot. Autorzy wykazali wyraźną dysproporcję w odporności narzędzi: podczas gdy zaawansowane filtry komercyjne zachowywały akceptowalną czułość, tradycyjne klasyfikatory statystyczne gwałtownie traciły skuteczność. Kluczowym wnioskiem z badań Alshammari i Rao było również wykazanie, że sam model językowy wykorzystany w trybie wnioskowania z łańcuchem myśli (*Chain-of-Thought*) oraz kilkoma przykładami referencyjnymi (*Few-Shot Prompting*) może przewyższać dedykowane detektory statystyczne, analizując logiczną spójność wywodu i subtelne relacje semantyczne.

Z kolei Wang i współpracownicy [**wang2023m4**], badając wielojęzyczny korpus M4, udowodnili, że metody tworzone z myślą o języku angielskim drastycznie tracą skuteczność po przeniesieniu na inne języki naturalne. W kontekście języka polskiego, charakteryzującego się bogatą fleksją i swobodnym szykiem zdania, luka badawcza jest szczególnie odczuwalna. Nowo powstające korpusy tekstowe, w tym SpeakLeash [**speakleash2024**], oraz polskie modele językowe, takie jak Bielik [**bielik2024**], otwierają możliwość systematycznego zbadania skuteczności znanych metod w polskim obszarze językowym, co stanowi bezpośrednią motywację dla niniejszego projektu.

2.2.7 Regulacje UE i komercyjne znakowanie wodne tekstu (2026)

Akt UE w sprawie sztucznej inteligencji (*EU AI Act*) zobowiązuje dostawców generatywnej AI do zapewnienia możliwości maszynowego rozpoznania wygenerowanego tekstu. W odpowiedzi na te wymagania OpenAI przedstawiło w październiku 2026 roku etapowe podejście do ustalania pochodzenia tekstu (*text provenance*) [**openai2026textprovenance**]. Firma udostępniła technologię znakowania wodnego pod nazwą **textGrain**, która osadza niewidoczny sygnał statystyczny w doborze słów przez model. Detektor OpenAI poszukuje tego sygnału i ocenia, czy dany fragment zawiera znak wodny pochodzący z systemów tej firmy.

Wdrożenie obejmuje trzy równoległe ścieżki. Po pierwsze, klienci API na całym świecie mogą dobrowolnie włączyć znakowanie wodne dla wybranych modeli (domyślnie funk-

cja pozostaje wyłączona). Po drugie, w Unii Europejskiej niewidoczny znak wodny ma być dodawany do kwalifikujących się odpowiedzi ChatGPT i Codex. Po trzecie, dostęp do detektora znaków wodnych jest początkowo ograniczony do zatwierdzonych badaczy i organizacji eksperckich, bez publicznego udostępnienia narzędzia przy premierze.

OpenAI podkreśla ograniczenia tej technologii. Przy docelowym odsetku wyników fałszywie dodatnich wynoszącym 1% detektor wykrywał znaki wodne w około 80% fragmentów o długości 200 tokenów oraz w około 95% fragmentów o długości 400 tokenów (dla treści z dziedziny psychologii). W dziedzinach o ograniczonym doborze słów, takich jak matematyka, wskaźniki wykrywalności były wyraźnie niższe. Edycja tekstu osłabia sygnał: w testach na fragmentach 400-tokenowych zastąpienie 10% słów synonimami obniżyło wykrywalność z około 92% do 66%, a zastąpienie 25% słów do około 17%.

Istotne są też granice interpretacyjne wyniku detekcji. Znak wodny nie mierzy udziału człowieka w redakcji tekstu, nie rozstrzyga o prawach autorskich ani odpowiedzialności, nie identyfikuje użytkownika ani polecenia i nie potwierdza prawdziwości treści. Brak wykrytego znaku wodnego nie dowodzi autorstwa człowieka: tekst mógł być zbyt krótki, zredagowany, przetłumaczony, pochodzić z nieobsługiwanego modelu albo z narzędzi innego dostawcy.

Dla niniejszego projektu wdrożenie textGrain ma dwojakie znaczenie. Po pierwsze, potwierdza, że aktywne znakowanie wodne po stronie generatora staje się elementem praktyki przemysłowej, ale obejmuje wyłącznie ekosystem konkretnego dostawcy. Po drugie, ograniczenia skuteczności (krótkie teksty, edycja, brak detekcji dla modeli zewnętrznych) wzmacniają potrzebę niezależnych metod detekcji post-hoc, które nie zakładają współpracy generatora. Właśnie w tym obszarze mieści się zakres planowanych prac własnych.

2.2.8 Zestawienie problemów badawczych i ewolucji rozwiązań

Podsumowanie przedstawionej historii rozwoju, ze szczególnym uwzględnieniem koncepcji poszczególnych autorów, zidentyfikowanych przez nich problemów oraz odpowiedzi formułowanych przez kolejnych badaczy, zestawiono w tabeli 1.

Rozwiązanie i autorzy	Główna koncepcja detekcji	Zidentyfikowane ograniczenia i problemy	Odpowiedź kolejnych badaczy
Klasyfikatory RoBERTa Solaiman i in. (2019) [**solaiman2019release**]	Nadzorowany fine-tuning modeli koderowych na parach tekstów ludzkich i z GPT-2.	Wysoki koszt anotacji zbalansowanych korpusów; podatność na przeuczenie i zjawisko przesunięcia domeny (*domain shift*).	Gehrmann i in. (2019) [**gehrmann2019gltr**] zaproponowali odejście od uczenia klasyfikatorów na rzecz bez-nadzorowej analizy statystyk modelu.
GLTR (analiza rang) Gehrmann, Strobelt, Rush (2019) [**gehrmann2019gltr**]	Inspekcja pozycji słów w rankingu predykcji generatora (nadmiar słów z czołówki rozkładu – tzw. „zielonych tokenów").	Wprowadzenie próbkowania stochastycznego (*nucleus / top-p sampling*) zatarło dyskretne rozkłady rangowe tokenów.	Tian (2023) [**gptzero2023**] połączył pomiar zakłopotania z analizą dynamiki składniowej tekstu w narzędziu GPTZero.
GPTZero (stylometria) Tian (2023) [**gptzero2023**]	Równoległa analiza zakłopotania (*perplexity*) oraz wariancji rytmu i długości zdań (*burstiness*).	Zawodzi na krótkich tekstach; Guo i in. (2023) [**guo2023hc3**] wykazali zacieranie się różnic stylometrycznych w domenach formalnych.	Mitchell i in. (2023) [**mitchell2023detectgpt**] przeszli od cech powierzchniowych do badania wewnętrznej geometrii prawdopodobieństwa.
DetectGPT Mitchell i in. (2023) [**mitchell2023detectgpt**]	Wykrywanie ujemnej krzywizny logarytmu prawdopodobieństwa wokół próbek sztucznych po perturbacjach semantycznych (model T5).	Wysoka złożoność obliczeniowa – konieczność generowania dziesiątek próbek perturbowanych dla każdego sprawdzanego tekstu.	Bao i in. (2024) [**bao2023fastdetectgpt**] wyprowadzili analityczną postać warunkowej krzywizny bez konieczności generowania próbek.
Fast-DetectGPT Bao i in. (2024) [**bao2023fastdetectgpt**]	Analityczne wyznaczanie warunkowej krzywizny prawdopodobieństwa tokenów; 300-krotne skrócenie czasu detekcji.	Wymaga dostępu do logitów modelu bazowego (*white-box*); podatność na ataki parafrazujące.	Kirchenbauer i in. (2023) [**kirchenbauer2023watermark**] zaproponowali alternatywę w postaci prewencyjnego znakowania wodnego.
Znakowanie wodne Kirchenbauer i in. (2023) [**kirchenbauer2023watermark**]	Pseudolosowy podział słownika na listę zieloną i czerwoną w trakcie próbkowania w oparciu o skrót poprzedniego tokena.	Wymaga pełnej współpracy dostawcy generatora; brak zastosowania do otwartych modeli pobieranych lokalnie.	Badacze skupili się na empirycznej weryfikacji ograniczeń metod post-hoc (Liang i in. [**liang2023gptdetectors**], Sadasivan i in. [**sadasivan2023can**]).
Granice detekcji Liang i in. (2023) [**liang2023gptdetectors**], Sadasivan i in. (2024) [**sadasivan2023can**]	Dowód dążenia AUC do 0,5 wraz z poprawą LLM; wykazanie dyskryminacji autorów obcojęzycznych (*non-native*).	Pasywna detekcja gotowego tekstu jest wysoce zawodna w warunkach polowych i podatna na zacieranie śladów przez parafrazę.	Alshammari i Rao (2025) [**alshammari2025evaluating**] zaproponowali wykorzystanie rozumowania samego modelu LLM (CoT / Few-Shot).
Wnioskowanie CoT Alshammari, Rao (2025) [**alshammari2025evaluating**]	Wykorzystanie zaawansowanego modelu LLM z technikami *Few-Shot* oraz *Chain-of-Thought* do jakościowej weryfikacji tekstu.	Wysoki koszt obliczeniowy dużych modeli; zależność od konstrukcji promptu; spadek stabilności w językach o mniejszych zasobach.	Konieczność badań nad lżejszymi, specyficznymi metodami w językach narodowych (Wang i in. [**wang2023m4**], projekt SpeakLeash [**speakleash2024**]).
textGrain (OpenAI) OpenAI (2026) [**openai2026textprovenance**]	Komercyjne znakowanie wodne tekstu pod wymogi *EU AI Act*; niewidoczny sygnał statystyczny w doborze słów; detektor dla badaczy.	Słaba odporność na edycję i tłumaczenie; spadek czułości na krótkich tekstach; brak pokrycia modeli innych firm; detektor niepubliczny.	Równoległa potrzeba niezależnej detekcji post-hoc (poza ekosystemem jednego dostawcy), zwłaszcza dla języka polskiego.

Tabela 1: Syntetyczne zestawienie ewolucji metod detekcji tekstu AI, ich problemów oraz odpowiedzi formułowanych przez kolejnych badaczy

2.3 Stan wiedzy

Problem sprowadza się zwykle do klasyfikacji binarnej: tekst ludzki vs tekst wygenerowany. Stosowane są dwa główne nurty:

1. **Podejścia statystyczne / stylometryczne** - entropia, perplexity, burstiness, rozkład długości zdań, częstości n-gramów. Działają bez dużego zbioru treningowego, ale bywają wrażliwe na domenę i długość tekstu.
2. **Uczenie maszynowe / głębokie** - klasyfikatory na embeddingach lub fine-tuning transformerów. Wymagają danych treningowych; przy małej liczbie przykładów łatwo o przeuczenie.

W literaturze podkreśla się, że metryki oparte na perplexity zawodzą przy bardzo krótkich próbkach (kilka zdań). Różnorodność generatorów (GPT, Llama, Gemini itd.) utrudnia generalizację jednego modelu detekcji.

2.4 Uwarunkowania komercyjne

Rynek detekcji AI jest związany głównie z edukacją, wydawnictwami i moderacją treści. Dostawcy komercyjni rzadko publikują pełne wyniki i ograniczenia modeli. Dla projektu akademickiego istotne są:

- dostępność otwartych zbiorów danych (Kaggle, Hugging Face),
- koszty API generatorów przy tworzeniu własnych próbek,
- brak gwarancji, że wynik detektora ma moc dowodową (ryzyko fałszywych oskarżeń).

2.5 Wyniki konsultacji z ekspertem

Konsultacje przeprowadzono z **dr. hab. Piotrem Hońko** (Wydział Informatyki PB). Poniżej pytania i odpowiedzi eksperta.

1. *Czy lepiej używać podejść statystycznych czy uczenia maszynowego?*
   Odpowiedź: Nie ma lepszego rozwiązania; dla małych zestawów danych podejście statystyczne może być lepsze.

2. *Czy teksty muszą być podobne tematycznie i czy w ogóle to jest możliwe?*
   Odpowiedź: Dla statystyki nie; dla ML tak.

3. *Jak podejść do problemu długości tekstu? Wiele metryk zawodzi na bardzo krótkich tekstach (np. 2-3 zdania).*
   Odpowiedź: Dobrać odpowiednie podejście.

4. *Czy warto nakładać minimalny limit długości tekstu w projekcie?*
   Odpowiedź: To zależy od podejścia - statystycznego albo ML.

2.6 Wyniki konsultacji AI

Te same pytania zadano modelowi językowemu (ChatGPT). Odpowiedzi AI:

1. *Podejścia statystyczne vs ML:*
   Odpowiedź AI: Przy ograniczonej liczbie przykładów warto zacząć od cech statystycznych (perplexity, burstiness, stylometria). ML/transformers mają potencjał wyższej dokładności, ale potrzebują większego, zróżnicowanego zbioru. Sensowna jest ścieżka hybrydowa: najpierw baseline statystyczny, potem klasyfikator ML.

2. *Podobieństwo tematyczne tekstów:*
   Odpowiedź AI: Homogeniczność tematyczna pomaga modelom ML (mniej zmiennych zakłócających). Metody statystyczne są bardziej odporne na zmianę domeny, ale i tak warto kontrolować źródło tekstów. Pełna niezależność od tematu jest trudna do osiągnięcia.

3. *Problem krótkich tekstów:*
   Odpowiedź AI: Dla 2-3 zdań należy unikać metryk wymagających stabilnego oszacowania rozkładu tokenów. Lepiej agregować cechy na poziomie dokumentu o ustalonej minimalnej długości albo stosować modele specjalnie do krótkich sekwencji. Alternatywnie - klasyfikować dłuższe fragmenty (akapity).

4. *Minimalny limit długości:*
   Odpowiedź AI: Tak, warto wprowadzić próg (np. liczba słów/tokenów), zwłaszcza przy metrykach statystycznych. Dla ML próg też pomaga, ale model może działać na krótszych próbkach kosztem pewności. Limit powinien wynikać z eksperymentów na wybranym zbiorze.

2.7 Porównanie odpowiedzi eksperta i AI

Pytanie	**dr hab. Piotr Hońko**	**Sztuczna inteligencja**
Statystyka vs ML	Brak jednego lepszego rozwiązania; przy małych danych statystyka może być lepsza.	Zacząć od baseline'u statystycznego; ML przy większych danych; możliwa hybryda.
Podobieństwo tematyczne	Dla statystyki nie jest konieczne; dla ML tak.	Homogeniczność pomaga ML; statystyka odporniejsza na domenę; pełna niezależność trudna.
Krótkie teksty	Dobrać odpowiednie podejście do problemu.	Unikać metryk niestabilnych na krótkich próbkach; agregować dłuższe fragmenty.
Limit długości	Zależy od wybranego podejścia (statystyka / ML).	Warto wprowadzić próg; dla statystyki szczególnie; próg ustalić eksperymentalnie.

Tabela 2: Porównanie konsultacji eksperckiej i AI

Wnioski z konsultacji: przy niepewności co do wielkości zbioru i realnej wykonalności zadania rozsądne jest rozpoczęcie od metod prostszych (statystycznych), z możliwością przejścia do ML, oraz świadome ograniczenie zakresu (m.in. długość tekstu).

3 Skrótowa analiza wymagań

Cel projektu: opracowanie metody (lub zestawu metod) pozwalającej odróżnić tekst napisany przez człowieka od tekstu wygenerowanego przez AI, z uwzględnieniem ograniczeń wynikających z konsultacji eksperckich oraz z obserwacji literatury (krótkie teksty, zmiana domeny, brak współpracy generatora).

3.1 Wymagania funkcjonalne

ID	Wymaganie	Priorytet
W1	Klasyfikacja binarna: tekst ludzki / tekst AI	wysoki
W2	Minimalna dokładność klasyfikacji $\geq 51\%$ na wybranym zbiorze testowym	wysoki
W3	Obsługa tekstów o ustalonej minimalnej długości (do ustalenia eksperymentalnie)	średni
W4	Porównanie co najmniej jednego podejścia statystycznego i jednego ML (o ile pozwolą dane)	średni
W5	Wykorzystanie otwartego zbioru danych (lub własnego podzbioru)	wysoki
W6	Udokumentowanie ograniczeń metody (krótkie teksty, domena, język)	średni
W7	Metoda działa bez dostępu do logitów generatora i bez znaku wodnego po stronie modelu (black-box / post-hoc)	wysoki
W8	Raportowanie podstawowych metryk poza dokładnością (precyzja, czułość lub F1)	średni

Tabela 3: Lista wymagań funkcjonalnych

3.2 Wymagania niefunkcjonalne

ID	Wymaganie	Priorytet
N1	Reprodukowalność eksperymentów (opis podziału danych, seed, wersje bibliotek)	wysoki
N2	Możliwość uruchomienia baseline'u na sprzęcie studenckim (CPU lub jeden GPU)	średni
N3	Kod i dokumentacja w repozytorium projektu	średni
N4	Świadome rozróżnienie detekcji post-hoc od detekcji znaku wodnego dostawcy (np. textGrain)	średni

Tabela 4: Lista wymagań niefunkcjonalnych

Poprzeczka dokładności została świadomie ustawiona nisko ($51\%+$), ponieważ na etapie planowania nie jest pewne, czy stabilna detekcja jest możliwa przy dostępnych zasobach i danych - zwłaszcza dla języka polskiego. Wymaganie W7 wynika z faktu, że komercyjne znakowanie wodne (np. OpenAI textGrain) obejmuje wyłącznie wybrane modele jednego dostawcy i nie zastępuje niezależnej detekcji tekstów z innych źródeł.

4 Zakres projektu i harmonogram

4.1 Zakres

W zakresie projektu:

- przegląd metod detekcji tekstu AI (statystyka, ML, watermarking),
- wybór i przygotowanie zbioru danych (podział train/test, filtr długości),
- implementacja baseline'u opartego na cechach statystycznych,
- implementacja prostego modelu ML (o ile pozwolą dane),
- ewaluacja: dokładność oraz podstawowe metryki jakości klasyfikacji,
- porównanie wyników z ograniczeniami opisanymi w literaturze,
- dokumentacja wyników, ograniczeń i wniosków.

Poza zakresem:

- detekcja w czasie rzeczywistym w produkcie komercyjnym,
- własne wdrażanie watermarkingu po stronie generatora,
- budowa lub hostowanie publicznego detektora znaków wodnych OpenAI (textGrain),
- pełna odporność na parafrazę, tłumaczenie i ataki adversarialne,
- gwarancja działania dla dowolnego języka i dowolnego modelu LLM,
- ocena prawna lub dowodowa wyniku detektora.

4.2 Proponowane zbiory danych

- **LLM Detect AI Generated Text** (Kaggle) - gotowy zbiór do detekcji tekstu AI (https://www.kaggle.com/datasets/sunilthite/llm-detect-ai-generated-
- **SpeakLeash (Spichlerz)** - duży korpus tekstów polskich; źródło tekstów ludzkich do zestawu PL (https://speakleash.org/).
- **HC3 / M4** - zbiory badawcze (głównie EN): human vs ChatGPT / multi-generator.

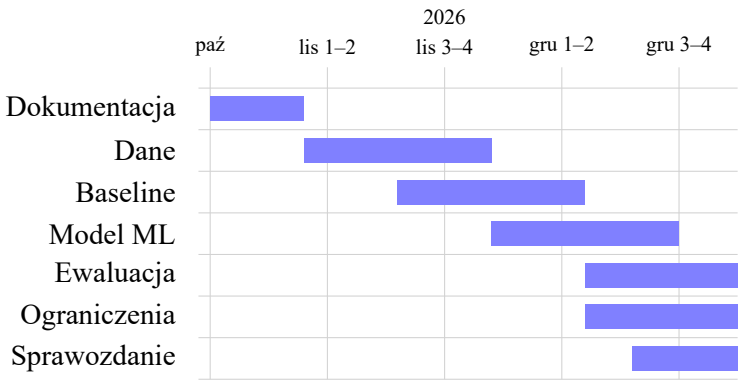
- **AIDE** - eseje uczniowskie vs LLM (kontekst edukacyjny).

Na starcie planowane jest wykorzystanie zbioru z Kaggle oraz - w miarę możliwości - fragmentów SpeakLeash do eksperymentów z językiem polskim.

4.3 Kamienie milowe

1. Zamknięcie przeglądu literatury i wymagań (październik 2026).
2. Przygotowanie zbioru danych i pipeline'u ewaluacji (listopad 2026).
3. Działający baseline statystyczny z wynikiem $\geq 51\%$ lub udokumentowaną przyczyną niepowodzenia (listopad–grudzień 2026).
4. Opcjonalny model ML oraz porównanie z baseline'em (grudzień 2026).
5. Sprawozdanie końcowe etapu z opisem ograniczeń (grudzień 2026).

4.4 Harmonogram (diagram Gantta)



Rysunek 4.1: Orientacyjny harmonogram prac (październik - grudzień 2026)

5 Szczegółowa specyfikacja projektu i opis wymagań

5.1 Cel i kryterium sukcesu

Sukces projektu definiowany jest jako uzyskanie klasyfikatora (lub metody statystycznej) osiągającego **dokładność co najmniej** 51% na wcześniej odłożonym zbiorze testowym, wraz z opisem warunków, w których wynik jest miarodajny (długość tekstu, język, źródło generatora).

Dodatkowym kryterium jakości jest udokumentowanie przypadków, w których metoda zawodzić będzie przewidywalnie (teksty krótkie, silna parafraza, mieszanka edycji ludzkiej i AI).

5.2 Wejście / wyjście

- **Wejście:** tekst w formie ciągłej (plik lub łańcuch znaków) spełniający minimalną długość.
- **Wyjście:** etykieta $\{\texttt{human}, \texttt{ai}\}$ oraz - jeśli to możliwe - wynik liczbowy (score / prawdopodobieństwo).
- **Metadane pomocnicze (opcjonalnie):** długość w tokenach/słowach, język, nazwa użytej metody.

5.3 Przepływ przetwarzania

1. Wczytanie tekstu i normalizacja (usunięcie zbędnych białych znaków, ewentualna detekcja języka).
2. Sprawdzenie minimalnej długości; poniżej progu zwracany jest komunikat o braku miarodajności zamiast twardej etykiety.
3. Ekstrakcja cech (statystycznych) albo wnioskowanie modelu ML.
4. Podjęcie decyzji klasyfikacyjnej według ustalonego progu.
5. Zapis wyniku i podstawowych metryk przy ewaluacji wsadowej.

5.4 Założenia

- Prace rozpoczynają się od podejścia statystycznego (zgodnie z konsultacją eksperta przy małej skali danych).
- Dla wariantu ML teksty treningowe powinny być możliwie zbliżone tematycznie (rekomendacja eksperta).
- Limit długości tekstu zostanie ustalony empirycznie.
- Projekt nie korzysta z prywatnego detektora znaków wodnych OpenAI; analizowana jest detekcja niezależna od współpracy generatora.
- Zbiór testowy nie jest używany do strojenia progów ani hiperparametrów.

5.5 Ograniczenia

- Dokładność 51% jest progiem minimalnym akceptacji, nie celem optymalnym.
- Wynik może nie przenosić się na inne domeny, modele LLM ani język polski, jeśli trening odbywał się głównie na danych angielskich.
- Metoda post-hoc nie zastępuje watermarkingu dostawcy i nie identyfikuje konkretnego modelu ani użytkownika.
- Brak odporności na celową parafrazę lub ciężką redakcję ludzką jest akceptowany na tym etapie.

5.6 Plan ewaluacji

- Podział danych: trening / walidacja / test (lub trening / test przy małym zbiorze).
- Metryki: dokładność; dodatkowo precyzja, czułość lub F1 dla klasy `ai`.
- Analiza wrażliwości na długość tekstu (kilka progów długości).
- Krótki opis błędów: przykłady fałszywie dodatnich i fałszywie ujemnych.

5.7 Środowisko realizacji

- Język implementacji: Python.
- Biblioteki: typowy stos analizy tekstu / ML (np. scikit-learn, transformers - zależnie od wariantu).
- Sprzęt: komputer studencki; ewentualnie darmowe środowisko notebookowe z GPU.

6 Opis wyników prac

Na obecnym etapie wykonano:

- sformułowanie tematu i zakresu projektu,
- przegląd podobnych rozwiązań i stanu wiedzy, w tym ewolucji metod detekcji,
- uzupełnienie tła o komercyjne znakowanie wodne OpenAI (textGrain) i kontekst *EU AI Act*,
- konsultacje z ekspertem (dr hab. Piotr Hońko) oraz z narzędziem GenAI,
- porównanie obu źródeł rekomendacji,
- uszczegółowienie wymagań funkcjonalnych i niefunkcjonalnych,
- doprecyzowanie zakresu, kamieni milowych i harmonogramu (diagram Gantta),
- szczegółową specyfikację wejścia/wyjścia, przepływu przetwarzania i planu ewaluacji,
- wstępną listę zbiorów danych (Kaggle, SpeakLeash i inne).

Implementacja metod detekcji oraz pomiary dokładności będą przedmiotem kolejnych etapów projektu.

6.1 Ścieżka machine learning

Zaimplementowano pipeline fine-tuningu modelu `distilbert-base-uncased` (67 mln parametrów) w PyTorch Lightning. Klasyfikacja jest binarna: `human` = 0, `ai` = 1. Eksperyment używa jednego seeda (`42`) i jednego podziału danych.

Czyszczenie danych i przeciek. Przed podziałem usunięto dokładne duplikaty oraz teksty krótsze niż 5 słów. Z 29 145 wierszy pozostało 27 296. Podział train/validation/test wynosi 21 838 / 2 729 / 2 729, a liczba wspólnych tekstów między splitami wynosi **0**. Wcześniejsze wyniki były zawyżone przez duplikaty: przed poprawką 10,4% testu miało dokładny odpowiednik w zbiorze treningowym.

Strojenie hiperparametrów. Uruchomiono jawną siatkę czterech konfiguracji. Zbiór testowy nie był używany podczas strojenia; wyboru dokonano na podstawie `val_f1`.

learning rate	batch size	max_length	best val F1	czas
2e-5	32	256	**0,99821**	186 s
3e-5	32	256	0,99821	195 s
4e-5	64	256	0,99821	212 s
2e-5	32	128	0,99599	157 s

Wybrano konfigurację **lr = 2e-5, batch size = 32, max_length = 256**. Liczba epok jest kontrolowana przez early stopping: próby mają limit 4 epok, a finalny trening — 20 epok z `patience = 5` na `val_f1`.

Wynik finalny. Po 12 epokach (najlepszy checkpoint z epoki 7) uzyskano na odłożonym zbiorze testowym:

metryka	wartość
accuracy	**0,99707**
precision	**1,00000**
recall	**0,99284**
F1	**0,99641**

Macierz pomyłek: `[[1611, 0], [8, 1110]]`, czyli 8 błędów na 2 729 próbek, wszystkie w stronę `ai` → `human`. Cały finalny trening zajął 8,4 min.

Stabilność i koszt. Krzywa uczenia jest stabilna: najgorszy epizod to epoka 3 (`val_f1` = 0,9933, `val_loss` = 0,0172). Fine-tuning wymaga GPU; na karcie RX 9070 XT (ROCm, bf16) jedna epoka zajmuje ok. 40 s, a pełny trening 8,4 min. Starszy wariant BERT-base we fp32 potrzebował ok. 9 min na epokę.

Zależność od długości tekstu. Średnia długość tekstu human to ok. 430 słów, a AI — ok. 312 słów. Sam klasyfikator oparty wyłącznie na liczbie słów osiąga AUC = 0,691, co oznacza, że długość jest istotnym confoundem. Dlatego metryki uzupełniono rozbiciem na kwartyle długości:

kwartyl (słowa)	próbki	accuracy	precision	recall	F1
Q1 (61–251)	692	0,99855	1,00000	0,99746	0,99873
Q2 (251–345)	678	0,99410	1,00000	0,98892	0,99443
Q3 (345–464)	677	0,99557	1,00000	0,98790	0,99391
Q4 (464–1436)	682	1,00000	1,00000	1,00000	1,00000

Ograniczenia metody ML. Najważniejsze ograniczenia to: koszt obliczeniowy fine-tuningu (wymagany GPU), overfitting (`train_loss` ok. $1{,}3 \cdot 10^{-6}$ przy `val_loss` ok. 0,006), zależność od domeny (tylko angielskie eseje z jednego zbioru, brak zewnętrznego testu), zależność od długości tekstu, pozostałe near-duplikaty oraz nierównowaga klas. Model nie był kalibrowany, a etykiety nie były weryfikowane ręcznie. Pełny opis znajduje się w dokumencie `train/ml-ograniczenia.md`.

Porównanie ze ścieżką statystyczną. Porównanie wymaga wyników ścieżki statystycznej przeliczonych na tym samym oczyszczonym zbiorze i tym samym podziale danych. Na moment przygotowania opisu takie zestawienie nie było jeszcze dostępne; bez wspólnego podziału porównanie nie byłoby metodologicznie poprawne.

Artefakty. Wagi finalnego modelu wyeksportowano do katalogu `models` (podkatalog `distilbert-final`) w formacie Hugging Face (`safetensors`). Konfiguracja, metryki i historia treningu znajdują się w katalogu `train/runs` (run `distilbert-final`), a skrypt eksportu to `train/export_model.py`.

7 Oświadczenie wykorzystania narzędzi GenAI

Autorzy oświadczają, że w przygotowaniu niniejszej dokumentacji skorzystano z narzędzi generatywnej AI (m.in. ChatGPT / asystent AI w środowisku Cursor) w zakresie:

- wygenerowania odpowiedzi AI do porównania z konsultacją ekspercką,
- pomocy redakcyjnej przy układzie i sformułowaniu dokumentacji,
- wsparcia przy składzie dokumentu w LaTeX.

Treści merytoryczne dotyczące wymagań, zakresu i interpretacji konsultacji eksperckich zostały zweryfikowane i zaakceptowane przez autorów. Odpowiedzi dr. hab. Piotra Hońko pochodzą z konsultacji przeprowadzonej przez zespół projektowy i nie były generowane automatycznie.

Michał Dąbrowski ..... data .....

Dominik Zabielski ..... data .....

Michał Niechciałkowski ..... data .....

Spis rysunków

4.1 Orientacyjny harmonogram prac (październik - grudzień 2026) 19